آموزش دستور شرطی ifدر پایتون

در برنامه‌نویسی، بسیاری از اوقات لازم است برنامه بر اساس یک **شرط** تصمیم‌گیری کند

برای مثال:

- اگر سن کاربر بیشتر از ۱۸ سال بود، پیام خاصی نمایش داده شود
- اگر نمره دانش‌آموز بیشتر از ۱۰ بود، اعلام شود که قبول شده است
- اگر رمز عبور درست بود، کاربر وارد حساب خود شود
- اگر عدد مثبت بود، پیام «عدد مثبت است» نمایش داده شود

برای انجام چنین کارهایی در پایتون از دستور شرطی ifاستفاده می‌کنیم

۱– شرط چیست؟

شرط در برنامه‌نویسی یک عبارت است که نتیجه آن معمولاً یکی از دو مقدار زیر است:

True یا False

یعنی یک شرط در نهایت می‌گوید: بله، این شرط برقرار است یا خیر، این شرط برقرار نیست.

برای مثال:

10 > 5

نتیجه:

True

زیرا ۱۰ بزرگ‌تر از ۵ است

اما:

10 < 5

نتیجه:

False

زیرا ۱۰ کوچک‌تر از ۵ نیست

۲– عملگرهای مقایسه‌ای در پایتون : برای ساختن شرط‌ها از Comparison Operatorsیا عملگرهای مقایسه‌ای استفاده می‌کنیم

عملگر	معنی	مثال	نتیجه
==	مساوی است	5 == 5	True
=!	مساوی نیست	5 =! 3	True
<	کوچک‌تر از	3 < 5	True
<=	کوچک‌تر یا مساوی	5 <= 5	True
>	بزرگ‌تر از	8 > 3	True
>=	بزرگ‌تر یا مساوی	8 >= 8	True

نکته بسیار مهم

در پایتون: = و== با یکدیگر تفاوت دارند ، علامت =برای انتساب مقدار استفاده می‌شود:

age = 20

یعنی مقدار ۲۰ را داخل متغیر ageقرار بده

اما ==برای **مقایسه** استفاده می‌شود:

age == 20

یعنی بررسی کن که آیا مقدار ageبرابر ۲۰ است یا خیر

۳– دستور if

ساختار کلی دستور ifبه شکل زیر است:

```
if condition:
    statement
```

در این ساختار:

- ifکلمه کلیدی پایتون است
- conditionهمان شرط است
- **:** نشان می‌دهد که دستورات مربوط به ifاز اینجا شروع می‌شوند
- دستور داخل ifباید دارای تورفتگی یا Indentation باشد

مثال:

```
age = 20
if age >= 18:
   print("You are an adult")
```

در این مثال ابتدا مقدار ageبرابر ۲۰ قرار گرفته است سپس برنامه بررسی می‌کند: age >= 18

یعنی: آیا ۲۰ بزرگ‌تر یا مساوی ۱۸ است؟

نتیجه Trueاست؛ بنابراین دستور print()اجرا می‌شود

خروجی:

You are an adult

۴– یک مثال ساده :

مثال زیر را ببینید:

```
a = 33
b = 200
if b > a:
   print("b is greater than a")
```

در اینجا:

```
a = 33
b = 200
```

دو متغیر داریم سپس شرط زیر بررسی می‌شود:

b > a

یعنی: آیا ۲۰۰ بزرگ‌تر از ۳۳ است؟

پاسخ:

True

بنابراین print()اجرا می‌شود

خروجی:

b is greater than a

۵– اگر شرط False باشد چه اتفاقی می‌افتد؟

اگر نتیجه شرط Falseباشد، دستورات داخل ifاجرا نمی‌شوند

مثلاً:

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
```

شرط این است:

b > a

یعنی:

آیا ۳۳ بزرگ‌تر از ۲۰۰ است؟

پاسخ: False

بنابراین دستور ()printاجرا نمی‌شود و برنامه هیچ خروجی‌ای تولید نمی‌کند

۷– مثال: بررسی مثبت بودن عدد

فرض کنید می‌خواهیم بررسی کنیم یک عدد مثبت است یا خیر

```
number = 15
if number > 0:
  print("The number is positive")
```

شرط:

number > 0

چون 0 < 15 درست است، پیام نمایش داده می‌شود:

The number is positive

۸– Indentation یا تورفتگی در پایتون

یکی از ویژگی‌های مهم پایتون این است که برای مشخص کردن محدوده دستورات از Indentationیا تورفتگی استفاده می‌کند

برای مثال:

```
age = 20
if age >= 18:
  print("You are an adult")
```

دستور print()با فاصله از ابتدای خط نوشته شده است این فاصله به پایتون می‌گوید که print()متعلق به ifاست

در بسیاری از زبان‌های برنامه‌نویسی مانندC ، C++و Java معمولاً از {}برای مشخص کردن محدوده دستورات استفاده می‌شود

اما پایتون از Indentationاستفاده می‌کند

۹– اشتباه رایج درIndentation

کد زیر اشتباه است:

```
a = 33
b = 200
if b > a:
print("b is greater than a")
```

چرا؟ زیرا دستور print()باید داخل بلوک ifقرار داشته باشد و بنابراین باید تورفتگی داشته باشد

شکل صحیح:

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```

۱۰– چند دستور داخل if

یک ifمی‌تواند شامل چندین دستور باشد برای مثال:

```
age = 20
if age >= 18:
   print("You are an adult")
   print("You can vote")
   print("You have full legal rights")
```

هر سه دستور دارای یک سطح تورفتگی هستند بنابراین هر سه متعلق به ifهستند اگر شرط Trueباشد، هر سه دستور اجرا می‌شوند.

خروجی:

```
You are an adult
You can vote
You have full legal rights
```

۱۱– قانون مهمIndentation

تمام دستوراتی که متعلق به یک بلوک هستند باید در یک سطح قرار داشته باشند

صحیح:

```
if age >= 18:
   print("Adult")
   print("You can vote")
   print("You can drive")
```

اما این کد اشتباه است:

```
if age >= 18:
   print("Adult")
     print("You can vote")
```

زیرا میزان تورفتگی دستورات یکسان نیست

۱۲– استفاده از متغیر Boolean در if

در پایتون نوع داده‌ای به نام Booleanیا boolداریم Booleanفقط دو مقدار دارد True و False

برای مثال:

```
is_logged_in = True
```

حالا می‌توانیم مستقیماً از این متغیر در ifاستفاده کنیم:

```
is_logged_in = True
if is_logged_in:
   print("Welcome back!")
```

چون مقدار is_logged_inبرابر Trueاست، پیام نمایش داده می‌شود

خروجی:

```
Welcome back!
```

۱۳– استفاده از Boolean برای تصمیم‌گیری

مثلاً:

```
is_student = True
if is_student:
   print("You are a student")
```

یا:

```
has_ticket = False
if has_ticket:
   print("You can enter")
```

در مثال دوم چون مقدار has_ticketبرابر Falseاست، دستور ()printاجرا نمی‌شود

۱۴– چه مقادیری در پایتون False محسوب می‌شوند؟

پایتون می‌تواند بسیاری از مقادیر را در یک شرط به Trueیا Falseتبدیل کند مقادیر مهمی که به صورت Falseدر نظر گرفته می‌شوند عبارت‌اند از:

False

0

""

None

همچنین مجموعه‌های خالی مانند:

[]

()

{}

نیز Falseمحسوب می‌شوند

مثال:

```
number = 0
if number:
   print("True")
```

چون numberبرابر صفر است، شرط اجرا نمی‌شود

۱۵– اعداد مثبت و منفی

اعداد غیرصفر معمولاً Trueمحسوب می‌شوند

مثلاً:

```
number = 5
if number:
   print("The value is True")
```

خروجی:

```
The value is True
```

عدد منفی نیز Trueمحسوب می‌شود:

```
number = -3
if number:
   print("The value is True")
```

بنابراین:

0 → False

5 → True

-3 → True

100 → True

۱۶– رشته‌های متنی و if

رشته خالی: "" برابر Falseمحسوب می‌شود

مثال:

```
name = ""
if name:
   print("Name exists")
```

در اینجا چیزی چاپ نمی‌شود اما اگر رشته دارای محتوا باشد:

```
name = "Ali"
if name:
   print("Name exists")
```

خروجی:

```
Name exists
```

حتی رشته‌ای مانند: "False" نیز Trueمحسوب می‌شود!

مثلاً:

```
value = "False"
if value:
   print("This is True")
```

خروجی:

```
This is True
```

زیرا "False"یک رشته غیرخالی است و با مقدار Boolean False تفاوت دارد

۱۷–تفاوت "False"و False

این نکته برای دانش‌آموزان بسیار مهم است

این: False یک مقدار Boolean است اما این "False" یک رشته متنی است

بنابراین:

```
if False:
   print("Hello")
```

چیزی چاپ نمی‌کند

اما:

```
if "False":
   print("Hello")
```

خروجی:

```
Hello
```

زیرا "False"یک رشته غیرخالی است

۱۸– جدول خلاصه مقادیر True وFalse

مقدار	نتیجه در شرط
True	True
False	False
0	False
1	True
-5	True
""	False
"Hello"	True
"False"	True
None	False
[]	False
[1, 2]	True
{}	False
{"name": "Ali"}	True

۱۹– یک مثال کاربردی: سیستم ورود

فرض کنیم می‌خواهیم بررسی کنیم آیا کاربر وارد سیستم شده است یا خیر:

```
is_logged_in = True

if is_logged_in:

   print("Welcome to the system")
```

اگر:

```
is_logged_in = False
```

باشد، پیام نمایش داده نمی‌شود ، این نوع ساختار در برنامه‌های واقعی بسیار کاربرد دارد

۲۰–یک مثال کاربردی: قبولی دانش‌آموز

```
score = 18

if score >= 10:

   print("You passed")
```

برنامه بررسی می‌کند:

score >= 10 چون نمره ۱۸ است، شرط درست است

خروجی:

You passed

۲۱– یک مثال کاربردی: بررسی سن

```
age = 25
if age >= 18:
   print("You are an adult")
```

اگر مقدار سن ۱۵ باشد:

```
age = 15
if age >= 18:
   print("You are an adult")
```

هیچ خروجی‌ای نمایش داده نمی‌شود؛ زیرا شرط False است

برنامه‌ای بنویسند که عدد زیر را بررسی کند:

number = 25

اگر عدد بزرگ‌تر از ۱۰ بود، پیام زیر نمایش داده شود:

The number is greater than 10

پاسخ:

```
number = 25
if number > 10:
   print("The number is greater than 10")
```

تمرین‌های کلاس

تمرین ۱

برنامه‌ای بنویسید که بررسی کند آیا عدد 50بزرگ‌تر از 20است یا خیر

تمرین ۲

برنامه‌ای بنویسید که سن فرد را دریافت کند و اگر سن او حداقل ۱۸ بود، پیام زیر را نمایش دهد:

You are an adult

تمرین ۳

برنامه‌ای بنویسید که نمره دانش‌آموز را بررسی کند و اگر نمره حداقل ۱۰ بود، عبارت زیر را نمایش دهد:

You passed

تمرین ۴

یک متغیر Boolean به نام زیر ایجاد کنید:

is_logged_in

اگر مقدار آن Trueبود، پیام زیر نمایش داده شود :

Welcome!

تمرین ۵

بررسی کنید که آیا یک رشته خالی است یا خیر:

name = ""

آموزش دستور elifدر پایتون :

با ifمی‌توانیم به برنامه بگوییم اگر یک شرط درست بود، دستورات مشخصی را اجرا کن اما گاهی اوقات فقط دو حالت نداریم و لازم است چند شرط مختلف را بررسی کنیم

برای مثال فرض کنید نمره یک دانش‌آموز را داریم:

- A → اگر نمره حداقل ۹۰ بود
- B → اگر نمره حداقل ۸۰ بود

- C → اگر نمره حداقل ۷۰ بود
- D → اگر نمره حداقل ۶۰ بود

برای چنین شرایطی می‌توانیم از elifاستفاده کنیم

۱– elif چیست؟

کلمه elifکوتاه‌شده‌ی else if است در پایتون از elifزمانی استفاده می‌کنیم که بخواهیم بعد از بررسی یک شرط، شرط دیگری را بررسی کنیم

به زبان ساده:

ifیعنی: اگر این شرط درست است، این کار را انجام بده

elifیعنی: اگر شرط قبلی درست نبود، این شرط را بررسی کن

ساختار کلی:

```
if condition1:
   statement1
elif condition2:
   statement2
```

۲–یک مثال ساده

مثال زیر را ببینید:

```
a = 33
b = 33
if b > a:
   print("b is greater than a")
elif a == b:
   print("a and b are equal")
```

ابتدا شرط ifبررسی می‌شود:

b > a یعنی آیا ۳۳ بزرگ‌تر از ۳۳ است؟

پاسخ:

False

بنابراین وارد قسمت elifمی‌شویم

شرط دوم:

a == b

یعنی آیا ۳۳ برابر ۳۳ است؟ پاسخ:

True

بنابراین خروجی برنامه بصورت زیر خواهد بود :

a and b are equal

۳– تفاوت ifو elif

این تفاوت بسیار مهم است

در ساختار:

```
if condition1:
   ...
elif condition2:
   ...
```

پایتون ابتدا شرط اول را بررسی می‌کند اگر شرط اول Trueباشد، همان قسمت اجرا می‌شود و دیگر elifبررسی نمی‌شود اگر شرط اول Falseباشد، پایتون سراغ elifمی‌رود

۴–چندین elif

در پایتون محدودیتی برای تعداد elifنداریم می‌توانیم چندین elifداشته باشیم:

```
if condition1:
   ...
elif condition2:
```

```
    ...
elif condition3:
    ...
elif condition4:
    ...
```

پایتون شرط‌ها را از بالا به پایین بررسی می‌کند به محض اینکه به اولین شرط Trueبرسد، همان بلوک را اجرا می‌کند و بررسی شرط‌های بعدی را متوقف می‌کند

۵–مثال: تعیین نمره

یکی از بهترین مثال‌ها برای آموزش elif، تعیین نمره دانش‌آموز است:

```
score = 75
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
```

مقدار:

```
score = 75
```

پایتون ابتدا بررسی می‌کند:

```
score >= 90
```

نتیجه:

```
False
```

سپس:

score >= 80

نتیجه:

False

سپس:

score >= 70

نتیجه:

True

بنابراین این دستور اجرا می‌شود:

print("Grade: C")

خروجی:

Grade: C

۶– نکته بسیار مهم: فقط اولین شرط درست اجرا می‌شود

فرض کنید:

```
score = 95
if score >= 90:
   print("Grade: A")
elif score >= 80:
   print("Grade: B")
elif score >= 70:
   print("Grade: C")
```

در این مثال، برای score = 95 هر سه شرط زیر از نظر منطقی درست هستند:

95 >= 90 → True

95 >= 80 → True

95 >= 70 → True

اما فقط این قسمت اجرا می‌شود:

print("Grade: A")

زیرا ifاولین شرط درست است پایتون بعد از اجرای آن دیگر سراغ elifهای بعدی نمی‌رود بنابراین خروجی فقط:

Grade: A

قانون مهم : در ساختار if / elif، فقط اولین شرطی که True باشد اجرا می‌شود

۷– ترتیب شرط‌ها بسیار مهم است

مثال زیر را ببینید:

```
score = 95
if score >= 70:
   print("Grade: C")
elif score >= 80:
   print("Grade: B")
elif score >= 90:
   print("Grade: A")
```

این برنامه برای نمره ۹۵ چه چیزی چاپ می‌کند؟

پاسخ:

Grade: C

چرا؟ زیرا پایتون ابتدا این شرط را بررسی می‌کند:

score >= 70

و چون درست است، بلافاصله آن را اجرا می‌کند بنابراین اصلاً به شرط‌های بعدی نمی‌رسد پس بهتر است شرط‌ها را با ترتیب منطقی مناسب بنویسیم:

```
if score >= 90:
   print("Grade: A")
elif score >= 80:
   print("Grade: B")
```

```
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
```

۸–مثال: دسته‌بندی سن

فرض کنید می‌خواهیم افراد را بر اساس سن دسته‌بندی کنیم:

```
age = 25
if age < 13:
    print("You are a child")
elif age < 20:
    print("You are a teenager")
elif age < 65:
    print("You are an adult")
elif age >= 65:
    print("You are a senior")
```

برای:

age = 25

پایتون ابتدا بررسی می‌کند:

age < 13

خیر

سپس:

age < 20

خیر

سپس:

age < 65

بله

بنابراین خروجی:

You are an adult

۹– چرا در این مثال شرط آخر می‌تواند ساده‌تر باشد؟

در مثال بالا داریم:

```
elif age >= 65:
   print("You are a senior")
```

این کد درست است، اما می‌توانیم از یک elseنیز استفاده کنیم:

```
if age < 13:
   print("You are a child")
elif age < 20:
   print("You are a teenager")
elif age < 65:
   print("You are an adult")
else:
   print("You are a senior")
```

در این حالت elseیعنی:

اگر هیچ‌کدام از شرط‌های قبلی درست نبودند، این قسمت را اجرا کن پس برای این مثال، استفاده از elseانتخاب مناسبی است

۱۰– if ، elifو else

سه دستور بسیار مهم برای تصمیم‌گیری در پایتون عبارت‌اند از:

if

elif

else

ساختار کلی:

```
if condition1:
   statement1
elif condition2:
   statement2
else:
   statement3
```

منطق آن:

اگر شرط اول درست بود

↓

اجرای if

در غیر این صورت اگر شرط دوم درست بود

↓

اجرای elif

در غیر این صورت

↓

اجرای else

مثال:

```
age = 15
if age >= 18:
   print("Adult")
elif age >= 13:
   print("Teenager")
else:
   print("Child")
```

خروجی:

Teenager

۱۱– مثال: تشخیص روز هفته

```
day = 3
if day == 1:
   print("Monday")
elif day == 2:
   print("Tuesday")
elif day == 3:
   print("Wednesday")
elif day == 4:
   print("Thursday")
elif day == 5:
   print("Friday")
elif day == 6:
   print("Saturday")
elif day == 7:
   print("Sunday")
```

چون:

day = 3

شرط زیر درست است:

day == 3

بنابراین خروجی بصورت زیر خواهد بود :

Wednesday

۱۲– مثال کاربردی: سیستم ورود

فرض کنید یک سیستم ساده داریم:

```
username = "ali"
password = "1234"
if username == "admin" and password == "1234":
   print("Admin login")
elif username == "ali" and password == "1234":
   print("Ali login")
elif username == "guest":
   print("Guest login")
```

در این مثال چند حالت مختلف را بررسی می‌کنیم

این موضوع در برنامه‌های واقعی بسیار کاربرد دارد؛ برای مثال:

- سیستم ورود کاربران
- سیستم ثبت‌نام
- فروشگاه اینترنتی
- سیستم نمرات
- برنامه‌های بانکی
- منوهای برنامه
- بازی‌ها

۱۳– تفاوت چند if با if / elif

این موضوع برای دانش‌آموزان بسیار مهم است

حالت اول: چند if

```
score = 95
if score >= 90:
   print("A")
if score >= 80:
   print("B")
if score >= 70:
   print("C")
```

در اینجا هر سه شرط جداگانه بررسی می‌شوند

بنابراین خروجی:

A

B

C

اما در :if / elif

```
score = 95
if score >= 90:
   print("A")
elif score >= 80:
   print("B")
elif score >= 70:
   print("C")
```

فقط:

A

چاپ می‌شود

نتیجه

اگر شرایط مستقل از یکدیگر باشند، چند ifممکن است مناسب باشد

اما اگر می‌خواهیم از بین چند حالت فقط یک حالت انتخاب شود، معمولاً از:

```
if
elif
elif
...
else
```

استفاده می‌کنیم

۱۴– چند elifمی‌توانیم داشته باشیم؟

هر تعداد که لازم باشد.

مثلاً:

```
if condition1:
    ...
elif condition2:
    ...
elif condition3:
    ...
elif condition4:
    ...
elif condition5:
    ...
```

اما باید توجه داشته باشیم که کد بیش از حد پیچیده نشود

۱۵–آیا elifحتماً باید بعد از ifباشد؟

بله نمی‌توانیم مستقیماً بنویسیم:

```
elif age > 18:
   print("Adult")
```

باید ابتدا یک ifداشته باشیم:

```
if age > 18:
   print("Adult")
elif age > 12:
   print("Teenager")
```

۱۶– آیا elifمی‌تواند بدون شرط باشد؟

خیر elifحتماً باید یک شرط داشته باشد:

```
elif age >= 18:
```

اما elseشرط ندارد:

```
else:
```

این تفاوت مهم است:

دستور	شرط دارد؟
if	بله
elif	بله
else	خیر

مثال :

```
number = 10
if number > 10:
   print("Greater than 10")
elif number == 10:
```

```
    print("Equal to 10")
else:
    print("Less than 10")
```

در این برنامه سه حالت داریم:

```
number > 10
```

یا:

```
number == 10
```

یا:

```
number < 10
```

اگر عدد ۱۰ باشد:

```
Equal to 10
```

اگر عدد ۲۰ باشد:

```
Greater than 10
```

و اگر عدد ۵ باشد:

```
Less than 10
```

مثال :

فرض کنید وارد یک فروشگاه شده‌اید و قیمت خرید شما مشخص است:

اگر خرید بیشتر از ۱,۰۰۰,۰۰۰ تومان بود ← تخفیف ۲۰٪

اگر خرید بیشتر از ۵۰۰,۰۰۰ تومان بود ← تخفیف ۱۰٪

اگر خرید بیشتر از ۲۰۰,۰۰۰ تومان بود ← تخفیف ۵٪

در غیر این صورت ← بدون تخفیف

```
price = 750000
if price >= 1000000:
   print("20% discount")
elif price >= 500000:
   print("10% discount")
elif price >= 200000:
   print("5% discount")
else:
   print("No discount")
```

برای :750000 ، نمایش داده می‌شود

10% discount

۱۹– اشتباه رایج :

یکی از اشتباهات رایج این است:

```
if score >= 90:
   print("A")
elif score >= 80
   print("B")
```

بعد از شرط elifنیز باید :قرار بگیرد:

```
elif score >= 80:
   print("B")
```

۲۰– اشتباه دوم: فراموش کردنIndentation

کد زیر اشتباه است:

```
score = 80
if score >= 90:
print("A")
```

```
elif score >= 80:
print("B")
```

شکل صحیح:

```
score = 80
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
```

تمرین مثبت، منفی یا صفر :

برنامه‌ای بنویسید که عدد زیر را بررسی کند:

number = -5

اگر مثبت بود:

Positive

اگر منفی بود:

Negative

و اگر صفر بود: Zero

تمرین نمره :

برنامه‌ای بنویسید که بر اساس نمره، رتبه دانش‌آموز را مشخص کند:

90 تا ۱۰۰ → A

80 تا ۸۹ → B

70 تا ۷۹ → C

60 تا ۶۹ → D

کمتر از ۶۰ → F

تمرین سن

برنامه‌ای بنویسید که سن فرد را دریافت کند:

0 تا ۱۲ → Child

13 تا ۱۹ → Teenager

20 تا ۶۴ → Adult

65 به بالا → Senior

تمرین روز هفته

متغیر زیر را در نظر بگیرید:

day = 5

با استفاده از if و elifمشخص کنید که روز پنجم هفته چه روزی است

تمرین سیستم تخفیف :

قیمت خرید را در متغیر زیر قرار دهید:

price = 850000

قوانین تخفیف:

1,000,000 و بیشتر → ۲۰٪

500,000 تا کمتر از ۱,۰۰۰,۰۰۰ → ۱۰٪

200,000 تا کمتر از ۵۰۰,۰۰۰ → ۵٪

کمتر از ۲۰۰,۰۰۰ → بدون تخفیف

مهم‌ترین نکته این است:

پایتون شرط‌ها را از بالا به پایین بررسی می‌کند و به محض پیدا کردن اولین شرط True، همان بلوک را اجرا کرده و بقیه شرط‌ها را بررسی نمی‌کند بنابراین هنگام استفاده از elif، ترتیب قرار گرفتن شرط‌ها بسیار مهم است

آموزش دستور elseدر پایتون :

در دو بخش قبلی با ifو elifآشنا شدیم. اکنون می‌خواهیم با دستور elseآشنا شویم. دستور elseزمانی استفاده می‌شود که هیچ‌کدام از شرط‌های قبلی درست نباشند

به زبان بسیار ساده:

ifیعنی «اگر این شرط درست بود«...

elifیعنی «اگر شرط قبلی درست نبود، این شرط را بررسی کن«...

elseیعنی «اگر هیچ‌کدام درست نبودند، این کار را انجام بده«.

۱– else چیست؟

کلمه کلیدی elseیک حالت پیش‌فرض ایجاد می‌کند

ساختار کلی:

```
if condition:
   statement1
else:
   statement2
```

۲–یک مثال ساده

```
a = 200
b = 33
if b > a:
   print("b is greater than a")
else:
   print("b is not greater than a")
```

شرط زیر بررسی می‌شود:

b > a

یعنی:

آیا ۳۳ بزرگ‌تر از ۲۰۰ است؟

پاسخ Falseاست

بنابراین ifاجرا نمی‌شود و برنامه وارد elseمی‌شود

خروجی:

b is not greater than a

۳– else بدون elif

برخلاف elif، برای استفاده از elseنیازی به داشتن elifنداریم

می‌توانیم فقط از:

if

else

استفاده کنیم

مثال:

```
age = 20
if age >= 18:
   print("You are an adult")
else:
   print("You are not an adult")
```

اگر ageبرابر ۲۰ باشد:

You are an adult

اما اگر:

age = 15

باشد، خروجی:

You are not an adult خواهد بود

۴– else یعنی حالت دوم

یکی از ساده‌ترین روش‌ها برای توضیح else این است که بگوییم:

if و else معمولاً یک تصمیم دوحالته ایجاد می‌کنند

مثلاً:

```
if condition:
   # حالت اول
else:
   # حالت دوم
```

یعنی:

اگر شرط درست است ⟶ حالت اول

در غیر این صورت ⟶ حالت دوم

مثلاً:

```
number = 10
if number > 0:
   print("Positive")
else:
   print("Not positive")
```

۵– مثال: عدد مثبت یا غیرمثبت

```
number = -5
if number > 0:
   print("The number is positive")
else:
   print("The number is not positive")
```

چون:

-5 > 0

برابر Falseاست، قسمت elseاجرا می‌شود

خروجی:

The number is not positive

۶– مثال بسیار مهم: عدد زوج و فرد

یکی از بهترین مثال‌ها برای آموزش else، تشخیص عدد زوج و فرد است در پایتون از عملگر %برای به‌دست آوردن باقی‌مانده تقسیم استفاده می‌کنیم

مثلاً: 2 % 10 نتیجه: 0

بنابراین اگر باقی‌مانده تقسیم یک عدد بر ۲ برابر صفر باشد، عدد زوج است

مثال:

```
number = 7
if number % 2 == 0:
   print("The number is even")
else:
   print("The number is odd")
```

برای عدد :7

7 % 2 = 1

پس شرط:

number % 2 == 0 برابر Falseاست بنابراین elseاجرا می‌شود:

The number is odd

۷– مثال با عدد زوج

اگر بنویسیم:

```
number = 8
if number % 2 == 0:
   print("The number is even")
else:
   print("The number is odd")
```

چون: 0 = 2 % 8 ، شرط درست است

بنابراین خروجی بصورت زیر خواهد بود :

The number is even

۸– else در کنار ifو elif

می‌توانیم if، elifو elseرا با هم استفاده کنیم

ساختار کلی:

```
if condition1:
   statement1
elif condition2:
   statement2
else:
   statement3
```

منطق:

اگر شرط اول درست بود

↓

اجرایif

اگر شرط اول درست نبود

↓

شرط دوم را بررسی کن

اگر شرط دوم هم درست نبود

↓

اجرایelse

۹– مثال: دما

فرض کنید می‌خواهیم دما را دسته‌بندی کنیم:

```
temperature = 22
if temperature > 30:
   print("It's hot outside!")
elif temperature > 20:
   print("It's warm outside")
elif temperature > 10:
   print("It's cool outside")
else:
   print("It's cold outside!")
```

مقدار دما:

temperature = 22

پایتون ابتدا بررسی می‌کند:

temperature > 30

نتیجه:

False

سپس:

temperature > 20

نتیجه:

True

بنابراین مقدار زیر چاپ می‌شود :

It's warm outside

۱۰– نقش elseدر این مثال

فرض کنید:

temperature = 5

پایتون بررسی می‌کند:

temperature > 30

خیر

سپس:

temperature > 20

خیر

سپس:

temperature > 10

خیر

هیچ‌کدام درست نبودند

بنابراین وارد elseمی‌شود:

else:

print("It's cold outside!")

خروجی:

It's cold outside!

۱۱– else به عنوان حالت پیش‌فرض

یکی از مهم‌ترین کاربردهای elseاین است که یک حالت پیش‌فرض ایجاد کنیم

مثلاً:

```
age = 70
if age < 13:
   print("Child")
elif age < 20:
   print("Teenager")
elif age < 65:
   print("Adult")
else:
   print("Senior")
```

در اینجا elseیعنی:

اگر هیچ‌کدام از شرط‌های قبلی درست نبودند، فرد Senior است

۱۲– else همیشه باید آخر باشد

این قانون بسیار مهم است

ساختار صحیح:

if condition1:

...

elif condition2:

...

else:

... اما نمی‌توانیم بعد از elseیک elifقرار دهیم

```
if age > 18:
   print("Adult")
else:
   print("Other")
elif age > 10:
   print("Teenager")
```

elseباید آخرین بخش زنجیره شرطی باشد

۱۳- مثال: قبولی یا مردودی

فرض کنید نمره دانش‌آموز را داریم:

```
score = 8
if score >= 10:
   print("You passed")
else:
   print("You failed")
```

چون:

8 =< 10

برابر Falseاست، خروجی:

You failed

خواهد بود

اگر:

score = 18

باشد:

You passed

چاپ می‌شود

۱۴– مثال: بررسی نام کاربری

یکی دیگر از کاربردهای else، اعتبارسنجی اطلاعات ورودی است

مثلاً:

```
username = "Ali"
if len(username) > 0:
   print(f"Welcome, {username}!")
else:
   print("Error: Username cannot be empty")
```

در اینجا:

len(username)

طول رشته را مشخص می‌کند چون usernameخالی نیست، شرط درست است

خروجی:

Welcome, Ali!

۱۶– اگر نام کاربری خالی باشد

اگر بنویسیم:

```
username = ""
if len(username) > 0:
   print(f"Welcome, {username}!")
else:
   print("Error: Username cannot be empty")
```

طول رشته برابر صفر است:

len(username) == 0

بنابراین شرط:

len(username) > 0

برابر Falseمی‌شود در نتیجه مقدار زیر نمایش داده می شود :

Error: Username cannot be empty

۱۷– می‌توان ساده‌تر نوشت

در پایتون، برای بررسی خالی نبودن یک رشته حتی می‌توانیم بنویسیم:

```
username = "Ali"
if username:
   print(f"Welcome, {username}!")
else:
   print("Error: Username cannot be empty")
```

اگر:

"" = username باشد، elseاجرا می‌شود این موضوع به مبحث قبلی ما درباره **Truthiness**مربوط است

۱۸– تفاوت if، elifو else

دستور	کاربرد
if	اولین شرط را بررسی می‌کند
elif	شرط دیگری را در صورت رد شدن شرط‌های قبلی بررسی می‌کند
else	زمانی اجرا می‌شود که هیچ‌کدام از شرط‌های قبلی درست نباشند

مثال:

```
score = 55
if score >= 90:
   print("A")
elif score >= 70:
   print("B")
```

```
elif score >= 60:
   print("C")
else:
   print("F")
```

چون هیچ‌کدام از شرط‌های بالا درست نیستند، elseاجرا می‌شود:

F

۱۹–یک مثال واقعی‌تر: سیستم تخفیف

فرض کنید یک فروشگاه برای مشتریان خود تخفیف در نظر گرفته است:

```
price = 150000
if price >= 1000000:
   print("20% discount")
elif price >= 500000:
   print("10% discount")
elif price >= 200000:
   print("5% discount")
else:
   print("No discount")
```

قوانین:

1,000,000به بالا ← ۲۰٪

500,000به بالا ← ۱۰٪

200,000به بالا ← ۵٪

کمتر از ۲۰۰,۰۰۰ ← بدون تخفیف

برای:

price = 150000

هیچ‌کدام از شرط‌ها درست نیستند

بنابراین:

No discount

نمایش داده می‌شود

۲۱– مثال: تشخیص سن

```
age = 8
if age >= 18:
   print("Adult")
elif age >= 13:
   print("Teenager")
else:
   print("Child")
```

چون سن ۸ است:

Adult → False

Teenager → False

پس:

Child

چاپ می‌شود

۲۱–مثال ترکیبی if + elif + else

مثال بسیار مناسب برای تمرین کلاسی:

```
score = 68
if score >= 90:
   print("Excellent")
elif score >= 70:
```

```
    print("Good")
elif score >= 60:
    print("Acceptable")
else:
    print("Failed")
```

برای نمره ۶۸:

score >= 90 → False

score >= 70 → False

score >= 60 → True

بنابراین:

Acceptable

چاپ می‌شود

آموزش Short Hand If در پایتون :

در مباحث قبلی با if، elifو elseآشنا شدیم. گاهی اوقات شرط ما بسیار ساده است و فقط **یک دستور** باید اجرا شود. در چنین شرایطی می‌توانیم کد را کوتاه‌تر بنویسیم به این روش معمولاً **Short Hand If**یا **One-Line If**گفته می‌شود.

همچنین در حالت if ... elseمی‌توانیم از **Conditional Expression**یا عبارت شرطی استفاده کنیم که در پایتون بسیار کاربردی است

۱- Short Hand If چیست؟

در حالت معمول، اگر بخواهیم بررسی کنیم که aاز bبزرگ‌تر است، می‌نویسیم:

```
a = 5
b = 2
if a > b:
    print("a is greater than b")
```

اما اگر فقط **یک دستور** داخل ifداشته باشیم، می‌توانیم آن را در همان خط بنویسیم:

```
a = 5
```

```
b = 2

if a > b: print("a is greater than b")
```

هر دو کد یک کار انجام می‌دهند

خروجی:

a is greater than b

۲–چرا به آن Short Hand می‌گوییم؟

کلمه‌ی **Short Hand** یعنی:

شکل کوتاه‌تر نوشتن یک دستور

در اینجا:

روش معمول

```
if condition:
    statement
```

روش کوتاه

```
if condition: statement
```

بنابراین می‌توانیم یک if ساده را در یک خط بنویسیم

۳– علامت :همچنان ضروری است

یکی از نکات مهم این است که حتی در حالت یک‌خطی، بعد از شرط if باید :قرار دهیم

صحیح:

```
if age >= 18: print("Adult")
```

اشتباه:

```
if age >= 18 print("Adult")
```

بنابراین ساختار:

```
if condition: statement
```

۴– مثال‌های ساده

مثال اول:

```
number = 10
if number > 0: print("Positive")
```

مثال دوم:

```
age = 20
if age >= 18: print("Adult")
```

مثال سوم:

```
score = 18
if score >= 10: print("Passed")
```

در هر سه مثال فقط یک دستور باید در صورت درست بودن شرط اجرا شود

۵– چه زمانی از One-Line if استفاده کنیم؟

وقتی شرط و دستور بسیار ساده باشند، استفاده از آن می‌تواند مناسب باشد

مثلاً:

```
if age >= 18: print("Adult")
```

خواندن این کد بسیار آسان است ، اما اگر چندین دستور داشته باشیم، بهتر است از شکل معمول استفاده کنیم:

```
if age >= 18:
   print("Adult")
   print("You can vote")
   print("You have full legal rights")
```

قرار دادن چنین کدی در یک خط، خوانایی برنامه را کاهش می‌دهد

۶– Short Hand if ... else

حالا فرض کنید دو حالت داریم:

```
a = 2
b = 330
if a > b:
   print("A")
else:
   print("B")
```

می‌توانیم این کد را به صورت کوتاه بنویسیم:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

خروجی:

```
B
```

۷– ساختار Short Hand if ... else

ساختار کلی:

```
value_if_true if condition else value_if_false
```

مثلاً:

```
print("A") if a > b else print("B")
```

یعنی: اگر a > b درست بود، "A" را چاپ کن؛ در غیر این صورت "B" را چاپ کن

۸– این ساختار چه نامی دارد؟

این روش در پایتون معمولاً Conditional Expression نامیده می‌شود گاهی نیز به آن:

Ternary Expression / Ternary Operator گفته می‌شود

عبارت شرطی کوتاه برای انتخاب بین دو مقدار یا دو نتیجه

۹– مثال ساده

```
age = 20
print("Adult") if age >= 18 else print("Not adult")
```

چون:

age >= 18

برابر Trueاست، خروجی:

Adult خواهد بود

اگر بنویسیم:

```
age = 15
print("Adult") if age >= 18 else print("Not adult")
```

خروجی:

Not adult

۱۰– یکی از مهم‌ترین کاربردها: اختصاص مقدار به متغیر

Conditional Expressionفقط برای print()نیست ، یکی از کاربردهای بسیار مهم آن، انتخاب یک مقدار و ذخیره آن در متغیر است

مثلاً:

```
a = 10
b = 20
bigger = a if a > b else b
print("Bigger is", bigger)
```

در اینجا:

bigger = a if a > b else b

یعنی:

اگر aبزرگ‌تر از bبود، مقدار aرا در biggerقرار بده؛ در غیر این صورت مقدار bرا قرار بده

چون:

10 > 20

غلط است، مقدار bانتخاب می‌شود

بنابراین:

Bigger is 20

۱۱– الگوی بسیار مهم

variable = value_if_true if condition else value_if_false

مثلاً:

result = "Pass" if score >= 10 else "Fail"

یعنی:

اگر نمره حداقل ۱۰ بود، Passرا در resultقرار بده؛ در غیر این صورت Fail

۱۲– مثال: قبولی دانش‌آموز

```
score = 15
result = "Passed" if score >= 10 else "Failed"
print(result)
```

خروجی:

Passed

اگر:

score = 8

باشد:

Failed

نمایش داده می‌شود

۱۳–مثال: بزرگ‌تر بودن دو عدد

فرض کنید:

x = 15

y = 20

می‌خواهیم عدد بزرگ‌تر را پیدا کنیم

روش معمول:

```
if x > y:
    max_value = x
else:
    max_value = y
```

روش کوتاه:

```
max_value = x if x > y else y
```

سپس:

```
print("Maximum value:", max_value)
```

خروجی:

Maximum value: 20

این یکی از کاربردهای بسیار خوب Conditional Expression است

۱۴– مثال: حداقل دو عدد

به همین شکل می‌توانیم کوچک‌ترین عدد را پیدا کنیم:

```
x = 15
y = 20
min_value = x if x < y else y
print("Minimum value:", min_value)
```

خروجی: Minimum value: 15

۱۵– مثال: تعیین وضعیت عدد

```
number = 10
result = "Even" if number % 2 == 0 else "Odd"
print(result)
```

اگر عدد زوج باشد:

Even

و اگر فرد باشد:

Odd

این مثال بسیار خوبی است، چون دانش‌آموزان همزمان با %و Conditional Expression تمرین می‌کنند

۱۶– مثال: تعیین بزرگسال یا کودک

```
age = 25
status = "Adult" if age >= 18 else "Child"
print(status)
```

خروجی:

Adult

۱۷– مثال: تعیین مثبت یا منفی

```
number = -5
result = "Positive" if number > 0 else "Not positive"
print(result)
```

خروجی:

Not positive

البته اگر بخواهیم سه حالت مثبت، منفی و صفر را بررسی کنیم، Conditional Expressionیک‌خطی دیگر خیلی خوانا نیست و بهتر است از if / elif / elseمعمولی استفاده کنیم

۱۸– چند شرط در یک خط

از نظر فنی می‌توان چند عبارت شرطی را نیز در یک خط قرار داد

مثلاً:

```
a = 330
b = 330
print("A") if a > b else print("=") if a == b else print("B")
```

۱۹–اما آیا این کار همیشه خوب است؟

خیر ، این کد از نظر پایتون معتبر است، اما خواندن آن برای یک دانش‌آموز مبتدی کمی دشوار است:

```
print("A") if a > b else print("=") if a == b else print("B")
```

بهتر است آن را به شکل زیر بنویسیم:

```
if a > b:
   print("A")
elif a == b:
   print("=")
else:
   print("B")
```

قانون مهم

کوتاه‌تر بودن کد همیشه به معنی بهتر بودن کد نیست هدف اصلی برنامه‌نویسی فقط کاهش تعداد خطوط نیست؛ خوانایی و قابل فهم بودن کد نیز بسیار مهم است

۲۰- تفاوت One-Line if و Conditional Expression

این دو مفهوم را بهتر است از هم جدا کنیم

One-Line if

فقط یک ifداریم:

```
if age >= 18: print("Adult")
```

یعنی: اگر شرط درست بود، این دستور را اجرا کن.

Conditional Expression

دو نتیجه داریم:

status = "Adult" if age >= 18 else "Child"

یعنی:

بر اساس شرط، یکی از دو مقدار را انتخاب کن

۲۱– مقایسه روش معمول و روش کوتاه

روش معمول

```
if score >= 10:
   result = "Passed"
else:
   result = "Failed"
```

روش کوتاه

result = "Passed" if score >= 10 else "Failed" هر دو یک نتیجه دارند اما در حالت دوم، کد کوتاه‌تر است

۲۲–مثال: نام کاربری پیش‌فرض

فرض کنید نام کاربری ممکن است خالی باشد:

username = ""

می‌خواهیم اگر نام کاربری وجود داشت، همان را نمایش دهیم و اگر خالی بود، Guestرا استفاده کنیم

می‌توانیم بنویسیم:

```
display_name = username if username else "Guest"
print("Welcome,", display_name)
```

خروجی:

Welcome, Guest

اگر بنویسیم:

username = "Ali"

خروجی:

Welcome, Ali

این یک کاربرد بسیار خوب از Conditional Expression است

۲۳– ارتباط با مفهوم Trueو False

در این مثال:

display_name = username if username else "Guest"

پایتون بررسی می‌کند که آیا usernameیک مقدار Truthyاست یا خیر

اگر:

username = "Ali"

باشد، رشته خالی نیست، بنابراین Trueمحسوب می‌شود

اما:

username = ""

رشته خالی است و Falseمحسوب می‌شود

بنابراین:

"Ali" → True

"" → False

۲۴– یک مثال کاربردی دیگر: قیمت

فرض کنید قیمت محصول را داریم price = 100 ، اگر قیمت بیشتر از ۵۰ باشد، وضعیت را Expensiveو در غیر این صورت Cheapقرار دهیم:

status = "Expensive" if price > 50 else "Cheap"

print(status)

خروجی: Expensive

۲۵–چه زمانی از Short Hand استفاده کنیم؟

استفاده از Short Hand زمانی مناسب است که:

- شرط ساده باشد.
- نتیجه ساده باشد.
- کد همچنان خوانا بماند
- فقط یک دستور یا یک انتخاب ساده داشته باشیم

مثلاً این کد مناسب است:

```
result = "Pass" if score >= 10 else "Fail"
```

یا:

```
if age >= 18: print("Adult")
```

۲۶–چه زمانی از Short Hand استفاده نکنیم؟

اگر منطق برنامه پیچیده باشد، بهتر است از ساختار معمول استفاده کنیم مثلاً به جای این کد پیچیده:

```
result = "A" if score >= 90 else "B" if score >= 80 else "C" if score >= 70 else "D"
```

بهتر است بنویسیم:

```
if score >= 90:
   result = "A"
elif score >= 80:
   result = "B"
elif score >= 70:
   result = "C"
else:
   result = "D"
```

نسخه دوم برای انسان بسیار راحت‌تر قابل خواندن و نگهداری است

آموزش عملگرهای منطقی در پایتونand — ، orو not

در مباحث قبلی یاد گرفتیم که چگونه با ifیک شرط را بررسی کنیم. اما در برنامه‌های واقعی، معمولاً لازم است چند شرط را همزمان بررسی کنیم

برای مثال: اگر سن فرد بیشتر از ۱۸ سال **و** دانشجو باشد، یک پیام نمایش بده

یا: اگر کاربر نام کاربری **یا** شماره تلفن معتبر داشته باشد، اجازه ادامه بده.

برای ترکیب شرط‌ها در پایتون از **Logical Operators**یا **عملگرهای منطقی** استفاده می‌کنیم

پایتون سه عملگر منطقی اصلی دارد:

عملگر	معنی	مفهوم ساده
and	و	همه شرط‌ها باید درست باشند
or	یا	حداقل یکی از شرط‌ها باید درست باشد
not	نقیض / معکوس	نتیجه شرط را برعکس می‌کند

۱–چرا به عملگر منطقی نیاز داریم؟

فرض کنید می‌خواهیم بررسی کنیم آیا یک فرد می‌تواند وارد یک سیستم شود یا خیر

ممکن است دو شرط داشته باشیم:

age >= 18

و:

is_student == True

اگر بخواهیم بگوییم فرد باید هم بالای ۱۸ سال باشد و هم دانشجو باشد از andاستفاده می‌کنیم:

```
if age >= 18 and is_student:
    print("Access granted")
```

۲– عملگر and

کلمه andبه معنی »و «است وقتی از andاستفاده می‌کنیم، تمام شرط‌ها باید Trueباشند تا نتیجه نهایی Trueشود

ساختار:

condition1 and condition2

یعنی: شرط اول و شرط دوم هر دو باید درست باشند

۳– مثال ساده and

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```

دو شرط داریم:

a > b

و:

c > a

بررسی کنیم:

200 > 33 → True

500 > 200 → True

هر دو شرط درست هستند بنابراین مقدار زیر نمایش داده می شود :

Both conditions are True

۴– اگر یکی از شرط‌ها False باشد چه می‌شود؟

مثلاً:

```
a = 200
b = 33
c = 100
```

```
if a > b and c > a:
    print("Both conditions are True")
```

شرط اول:

200 > 33 → True

شرط دوم:

100 > 200 → False

بنابراین:

True and False

برابر Falseاست

پس print()اجرا نمی‌شود

۵– جدول درستی and

برای یادگیری andاین جدول بسیار مهم است:

and	شرط دوم	شرط اول
True	True	True
False	False	True
False	True	False
False	False	False

قانون ساده

در and، همه شرط‌ها باید درست باشند .

۶– مثال واقعی and

فرض کنید برای ورود به یک مسابقه فرد باید حداقل ۱۸ سال داشته باشد و ثبت‌نام هم کرده باشد:

```
age = 20
is_registered = True
if age >= 18 and is_registered:
    print("You can enter the competition")
```

هر دو شرط درست هستند:

age >= 18 → True

is_registered → True

بنابراین:

You can enter the competition

۷–مثال: دسترسی به سیستم

فرض کنید کاربر فقط در صورتی اجازه ورود دارد که:

۱. رمز عبور درست باشد.

۲. حساب کاربری فعال باشد.

```
correct_password = True
account_active = True
if correct_password and account_active:
    print("Access granted")
```

اگر یکی از آنها Falseباشد:

```
correct_password = True
account_active = False
```

شرط نهایی Falseمی‌شود

۸–عملگر or

کلمه orبه معنی **»یا «**است در orکافی است حداقل یکی از شرط‌ها Trueباشد

ساختار:

condition1 or condition2

یعنی:

اگر شرط اول یا شرط دوم درست بود، نتیجه درست است

۹– مثال ساده or

```
a = 200
b = 33
c = 500
if a > b or a > c:
  print("At least one of the conditions is True")
```

شرط اول:

200 > 33 → True

شرط دوم:

200 > 500 → False

بنابراین:

True or False

نتیجه: True است . پس پیام نمایش داده می‌شود.

۱۰– جدول درستی or

شرط اول	شرط دوم	or
True	True	True
True	False	True
False	True	True
False	False	False

قانون ساده

در orکافی است حداقل یکی از شرط‌ها درست باشد

۱۱– مثال واقعی or

فرض کنید برای ورود به یک ساختمان فرد می‌تواند با کارت دانشجویی **یا** کارت کارمندی وارد شود

```
has_student_card = False
has_employee_card = True
```

```
if has_student_card or has_employee_card:
    print("Access granted")
```

چون حداقل یکی از شرط‌ها Trueاست، ورود مجاز است

خروجی:

Access granted

۱۲– تفاوت andو or

and

True and True → True

ولی:

True and False → False

or

True or False → True

اما:

False or False → False

به صورت خیلی ساده:

AND → باشند True همه باید

OR → باشد True حداقل یکی باید

۱۳– عملگر not

سومین عملگر منطقی notاست ، notنتیجه یک شرط را **معکوس** می‌کند

یعنی:

True → False

False → True

مثلاً:

```
a = 33
b = 200
if not a > b:
  print("a is NOT greater than b")
```

ابتدا:

a > b

بررسی می‌شود

یعنی:

33 > 200 → False

اما notنتیجه را معکوس می‌کند:

not False → True

بنابراین پیام چاپ می‌شود:

a is NOT greater than b

۱۴– جدول درستی not

شرط	notشرط
True	False
False	True

قانون ساده

notنتیجه را برعکس می‌کند

مثلاً:

```
is_logged_in = True
if not is_logged_in:
  print("Please login")
```

چون:

```
is_logged_in = True
```

است:

not True → False

بنابراین پیام چاپ نمی‌شود

۱۵–مثال کاربردی not

```
is_raining = False
if not is_raining:
    print("You can go outside")
```

چون:

is_raining = False

بنابراین:

not False → True

و پیام نمایش داده می‌شود

۱۶– ترکیب and، or و not

می‌توانیم چند عملگر منطقی را با یکدیگر ترکیب کنیم

مثلاً:

```
age = 25
is_student = False
has_discount_code = True
if (age < 18 or age > 65) and not is_student or has_discount_code:
    print("Discount applies!")
```

در این مثال چند شرط داریم

قسمت اول:

age < 18 or age > 65

قسمت دوم:

```
not is_student
```

و در نهایت:

```
has_discount_code
```

بهتر است در چنین شرایطی برای خوانایی بیشتر از پرانتز استفاده کنیم

۱۷– چرا از پرانتز استفاده کنیم؟

وقتی چند عملگر منطقی داریم، ممکن است خواندن کد دشوار شود

مثلاً:

```
if age > 18 and is_student or has_discount:
```

برای برنامه‌نویس مشخص کردن منظور کمی سخت‌تر است

بهتر است بنویسیم:

```
if (age > 18 and is_student) or has_discount:
   print("Discount applies")
```

پرانتز مشخص می‌کند که ابتدا کدام قسمت را به عنوان یک گروه منطقی در نظر گرفته‌ایم

۱۸- اولویت عملگرهای منطقی

اگر چند عملگر منطقی در یک عبارت وجود داشته باشند، پایتون ترتیب مشخصی برای ارزیابی آنها دارد:

not

↓

and

↓

or

یعنی به طور کلی:

ابتدا not و سپس and و سپس or ارزیابی می‌شود

مثلاً:

True or False and False

ابتدا andبررسی می‌شود:

False and False → False

سپس:

True or False → True

پس نتیجه نهایی:

True

با این حال برای کدهای آموزشی و پیچیده بهتر است از پرانتز استفاده کنیم

۱۹–مثال با پرانتز

```
temperature = 25
is_raining = False
is_weekend = True
if (temperature > 20 and not is_raining) or is_weekend:
    print("Great day for outdoor activities!")
```

ابتدا:

temperature > 20

نتیجه:

True

و:

not is_raining

چون is_rainingبرابر Falseاست:

not False → True

بنابراین:

True and True → True

و در نهایت:

True or True → True

پس پیام چاپ می‌شود

۲۰– مثال کاربردی: اعتبارسنجی کاربر

فرض کنید برای ورود به سیستم سه مورد را بررسی می‌کنیم:

```
username = "Ali"
password = "1234"
is_verified = True
if username and password and is_verified:
   print("Login successful")
else:
   print("Login failed")
```

در اینجا سه شرط داریم: username و password و: is_verified

چون هر سه مقدار درست و غیرخالی هستند، شرط andبرقرار است

خروجی:

Login successful

۲۱– چرا usernameو passwordرا مستقیماً استفاده کردیم؟

این نکته به مبحث قبلی **Truthiness**مربوط می‌شود

مثلاً:

username = "Ali"

رشته خالی نیست، بنابراین در یک شرط Trueمحسوب می‌شود

اما: username = "" ، رشته خالی است و Falseمحسوب می‌شود

بنابراین:

```
if username:
    print("Username exists")
```

یعنی:

اگر usernameخالی نیست، این دستور را اجرا کن

۲۲– مثال: بررسی محدوده نمره

فرض کنید نمره باید بین ۰ و ۱۰۰ باشد می‌توانیم بنویسیم:

```
score = 85
if score >= 0 and score <= 100:
    print("Valid score")
else:
    print("Invalid score")
```

چون:

85 >= 0 → True

85 <= 100 → True

بنابراین:

Valid score

نمایش داده می‌شود

۲۳–روش ساده‌تر برای بررسی محدوده

در پایتون می‌توانیم این شرط را حتی کوتاه‌تر بنویسیم:

```
score = 85
if 0 <= score <= 100:
    print("Valid score")
else:
    print("Invalid score")
```

این یکی از ویژگی‌های زیبای پایتون است که به آن Chained Comparisonگفته می‌شود

یعنی:

```
0 <= score <= 100
```

به معنی:

scoreحداقل ۰ و حداکثر ۱۰۰ باشد

۲۴– مثال واقعی: ورود به سایت

فرض کنید شرایط ورود این است:

- نام کاربری خالی نباشد
- رمز عبور خالی نباشد
- حساب کاربر تأیید شده باشد

```
username = "Ali"
password = "1234"
is_verified = True
if username and password and is_verified:
   print("Login successful")
else:
   print("Login failed")
```

در اینجا andبسیار مناسب است، زیرا هر سه شرط باید برقرار باشند

۲۵– مثال: روش‌های مختلف ورود

فرض کنید کاربر می‌تواند با:

- ایمیل
- یا شماره تلفن

وارد شود

```
has_email = False
has_phone = True
if has_email or has_phone:
    print("You can continue")
else:
    print("Please provide email or phone")
```

چون حداقل یکی از دو شرط Trueاست، ورود امکان‌پذیر است

۲۶ مثال :notعدم ورود کاربر

```
is_logged_in = False
if not is_logged_in:
    print("Please login first")
```

چون:

is_logged_in = False

داریم:

not False → True

پس:

Please login first

نمایش داده می‌شود

۲۷– تفاوت مهم =!و not

دانش‌آموزان ممکن است این دو را با هم اشتباه بگیرند

=!یعنی: مساوی نیست

مثلاً:

age =! 18 اما notنتیجه یک عبارت را معکوس می‌کند:

not age == 18

در بسیاری از شرایط این دو نتیجه منطقی مشابهی دارند:

age != 18

و:

not (age == 18)

اما مفهوم آنها دقیقاً یکی نیست؛ notیک عبارت منطقی را معکوس می‌کند

۲۸–مثال ترکیبی

فرض کنید تخفیف در یکی از این شرایط اعمال شود:

- سن کمتر از ۱۸ سال باشد
- سن بیشتر از ۶۵ سال باشد
- کاربر کد تخفیف داشته باشد

```
age = 25
has_discount_code = True
if age < 18 or age > 65 or has_discount_code:
   print("Discount applies")
else:
   print("No discount")
```

چون:

age < 18 → False

age > 65 → False

has_discount_code → True

حداقل یکی از شرایط Trueاست

بنابراین:

Discount applies

۲۹– جدول مقایسه سه عملگر

عملگر	معنی	چه زمانی **True** می‌شود؟
and	و	وقتی همه شرط‌ها True باشند
or	یا	وقتی حداقل یک شرط True باشد
not	معکوس	نتیجه شرط را برعکس می‌کند

مثال:

True and True

نتیجه: True

True or False

نتیجه: True

not True

نتیجه: False

تمرین ۱:

برنامه‌ای بنویسید که بررسی کند:

- سن حداقل ۱۸ باشد
- فرد دانشجو باشد

age = 20

is_student = True

اگر هر دو شرط برقرار بود، بنویسد:

Eligible

تمرین ۲ :

برنامه‌ای بنویسید که اگر فرد کارت دانشجویی یا کارت کارمندی داشت، اجازه ورود بدهد

has_student_card = False

has_employee_card = True

تمرین ۳ :

متغیر زیر را در نظر بگیرید:

is_raining = False

اگر باران نمی‌بارد، پیام زیر را چاپ کنید:

You can go outside

تمرین ۴ : محدوده نمره

برنامه‌ای بنویسید که بررسی کند نمره بین ۰ و ۱۰۰ باشد

score = 75

تمرین ۵ : ورود به سیستم

سه متغیر داریم:

username = "Ali"

password = "1234"

is_verified = True

اگر هر سه معتبر بودند:

Login successful

در غیر این صورت:

Login failed

تمرین ۶ : تخفیف

اگر یکی از شرایط زیر برقرار بود، تخفیف بدهید:

- سن کمتر از ۱۸ باشد
- سن بیشتر از ۶۵ باشد
- کد تخفیف داشته باشد

age = 25

has_discount_code = True

آموزش Nested If در پایتون :

در مباحث قبلی با if، elif، elseو همچنین عملگرهای منطقی مانند and، orو notآشنا شدیم ، گاهی اوقات لازم است بعد از درست بودن یک شرط، یک شرط دیگر را بررسی کنیم .در این حالت می‌توانیم یک ifرا داخل ifدیگری قرار دهیم

به این ساختار Nested Ifیا شرط تو در تو گفته می‌شود

۱– Nested If چیست؟

کلمه‌ی **Nested**به معنی «تو در تو» است در پایتون می‌توانیم یک ifرا داخل یک ifدیگر قرار دهیم:

```
if condition1:
   if condition2:
      statement
```

یعنی:

ابتدا شرط اول بررسی می‌شود؛ اگر شرط اول درست بود، شرط دوم بررسی می‌شود

۲– یک مثال بسیار ساده

```
x = 41
if x > 10:
   print("Above ten")
   if x > 20:
      print("and also above 20!")
```

ابتدا پایتون بررسی می‌کند:

x > 10

چون:

41 > 10 → True

وارد ifاول می‌شود

سپس ifداخلی را بررسی می‌کند:

x > 20

چون:

41 > 20 → True

پیام دوم نیز نمایش داده می‌شود

خروجی:

Above ten

and also above 20!

۳– نکته بسیار مهم

ifداخلی فقط زمانی اجرا می‌شود که ifخارجی برقرار باشد

مثلاً:

```
x = 5
if x > 10:
   print("Above ten")
   if x > 20:
      print("Above twenty")
```

شرط اول:

5 > 10 → False است بنابراین پایتون اصلاً وارد ifداخلی نمی‌شود در نتیجه هیچ خروجی‌ای نداریم

قانون مهم:

شرط داخلی وابسته به برقرار بودن شرط خارجی است

۴- ساختار کلیNested If

ساختار اصلی:

if condition1:

statement1

if condition2:

statement2

دقت کنید که ifدوم دارای Indentationبیشتری است

این فاصله به پایتون می‌گوید که ifدوم داخل ifاول قرار دارد

۵–نقش Indentation درNested If

به این کد دقت کنید:

```
if age >= 18:
   print("You are an adult")
   if has_license:
      print("You can drive")
```

if has_licenseداخل if age >= 18است

اما این کد:

```
if age >= 18:
   print("You are an adult")
if has_license:
   print("You can drive")
```

دیگر Nested If نیست در اینجا دو ifمستقل داریم.

۶– مثال: سن و گواهینامه

فرض کنید می‌خواهیم بررسی کنیم آیا فرد می‌تواند رانندگی کند یا خیر

شرط اول:

آیا فرد حداقل ۱۸ سال دارد؟ اگر بله، شرط دوم را بررسی می‌کنیم:

آیا گواهینامه دارد؟

```
age = 25
```

```
has_license = True
if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("You are too young to drive")
```

۷– مرحله‌به‌مرحله اجرای مثال

مقدارها:

```
age = 25
has_license = True
```

ابتدا:

```
age >= 18
```

بررسی می‌شود:

25 >= 18 → True

پس وارد ifداخلی می‌شویم

سپس:

```
has_license
```

بررسی می‌شود:

True

بنابراین نمایش داده می شود :

You can drive

۸– اگر فرد گواهینامه نداشته باشد

اگر:

```
has_license = False
```

باشد:

```
age = 25
has_license = False
if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("You are too young to drive")
```

نتیجه:

```
You need a license
```

۹– اگر فرد کمتر از ۱۸ سال باشد

مثلاً:

```
age = 16
has_license = True
```

شرط اول:

16 >= 18 → False

بنابراین if داخلی اصلاً اجرا نمی‌شود و نتیجه به صورت زیر خواهد بود:

```
You are too young to drive
```

حتی اگر:

has_license = True

باشد، چون شرط خارجی برقرار نیست، شرط داخلی بررسی نمی‌شود

۹– Nested If با سه سطح

در پایتون می‌توان چند سطح ifتو در تو داشت

مثلاً:

```
score = 85
attendance = 90
submitted = True
if score >= 60:
    if attendance >= 80:
        if submitted:
            print("Pass with good standing")
        else:
            print("Pass but missing assignment")
    else:
        print("Pass but low attendance")
else:
    print("Fail")
```

۱۱– نحوه اجرای سه سطح

مقدارها:

score = 85

attendance = 90

submitted = True

مرحله اول:

score >= 60

نتیجه:

True

مرحله دوم:

attendance >= 80

نتیجه:

True

مرحله سوم:

submitted

نتیجه:

True

پس خروجی یه صورت زیر خواهد بود:

Pass with good standing

۱۲–اگر تکلیف تحویل داده نشده باشد

اگر: submitted = False باشد:

score >= 60 → True

attendance >= 80 → True

submitted → False

بنابراین بصورت زیر نمایش داده می شود :

Pass but missing assignment

۱۳– اگر حضور و غیاب کم باشد

مثلاً:

attendance = 70

آنگاه:

score >= 60 → True

attendance >= 80 → False

پایتون دیگر وارد if submittedنمی‌شود

خروجی به صورت زیر است :

Pass but low attendance

۱۴– Nested If و and

گاهی اوقات Nested If را می‌توان با andساده‌تر کرد مثلاً:

Nested If

```
temperature = 25
is_sunny = True
if temperature > 20:
    if is_sunny:
        print("Perfect beach weather!")
```

می‌توانیم آن را به صورت زیر بنویسیم:

```
temperature = 25
is_sunny = True
if temperature > 20 and is_sunny:
    print("Perfect beach weather!")
```

هر دو کد در این مثال نتیجه یکسانی دارند

۱۵–چه زمانی Nested If استفاده کنیم؟

اگر دو شرط ساده و مستقل داشته باشیم، معمولاً andخواناتر است:

```
if age >= 18 and has_license:
    print("You can drive")
```

اما گاهی شرط دوم فقط زمانی معنی دارد که شرط اول برقرار باشد

در این شرایط Nested If می‌تواند مناسب‌تر باشد:

```
if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("You are too young")
```

در این حالت ساختار تصمیم‌گیری واضح‌تر است

۱۶– تفاوت مفهومی Nested Ifوand

این دو کد:

Nested If

```
if temperature > 20:
    if is_sunny:
        print("Great weather")
```

and

```
if temperature > 20 and is_sunny:
    print("Great weather")
```

در این مثال نتیجه یکسان است

اما Nested If به ما اجازه می‌دهد برای هر مرحله تصمیم جداگانه‌ای داشته باشیم

مثلاً:

```
if temperature > 20:
    if is_sunny:
        print("Great weather")
    else:
```

```
    print("Warm but cloudy")
else:
  print("Temperature is low")
```

این ساختار با یک andساده به همین شکل قابل بیان نیست

۱۷–مثال کاربردی: ورود به سیستم

فرض کنید سیستم ورود سه مرحله دارد:

۱. آیا username وارد شده است؟

۲. آیا password وارد شده است؟

۳. آیا حساب فعال است؟

```
username = "Ali"
password = "python123"
is_active = True
if username:
  if password:
    if is_active:
      print("Login successful")
    else:
      print("Account is not active")
  else:
    print("Password required")
else:
  print("Username required")
```

۱۸– نحوه اجرای سیستم ورود

فرض کنید: username = "" باشد

شرط: if username: برابر Falseمی‌شود

بنابراین خروجی بصورت زیر است :

Username required

پایتون حتی passwordو is_activeرا بررسی نمی‌کند

۱۹– اگر username درست باشد ولی password خالی باشد

username = "Ali"

password = ""

is_active = True

ابتدا:

username → True

پس وارد شرط دوم می‌شویم.

اما:

password → False

بنابراین:

Password required

نمایش داده می‌شود

۲۰– اگر حساب غیرفعال باشد

username = "Ali"

password = "python123"

is_active = False

در این حالت دو شرط اول درست هستند، اما شرط سوم:

is_active → False

پس:

Account is not active

نمایش داده می‌شود

۲۱–مثال: سیستم نمره‌دهی

فرض کنید:

score = 92

extra_credit = 5

می‌خواهیم اگر نمره حداقل ۹۰ بود، بررسی کنیم آیا امتیاز اضافه دارد یا خیر

```
if score >= 90:
    if extra_credit > 0:
        print("A+ grade")
    else:
        print("A grade")
elif score >= 80:
    print("B grade")
else:
    print("C grade or below")
```

چون:

92 >= 90 → True

و:

5 > 0 → True

خروجی:

A+ grade

۲۲– Nested If با elif

نکته مهم این است که Nested If فقط شامل ifداخل ifنیست. می‌توانیم در سطوح مختلف از elifو elseنیز استفاده کنیم

مثلاً:

```
age = 25
if age >= 18:
   if age >= 65:
      print("Senior adult")
   elif age >= 30:
      print("Adult")
   else:
      print("Young adult")
else:
   print("Minor")
```

اینجا elifداخل ifخارجی قرار گرفته است

۲۳– Nested If بیش از حد

از نظر فنی می‌توانیم تعداد زیادی ifرا داخل یکدیگر قرار دهیم:

```
if condition1:
   if condition2:
      if condition3:
         if condition4:
            if condition5:
               print("...")
```

اما این کار معمولاً باعث می‌شود برنامه:

- سخت خوانده شود
- سخت اشکال‌زدایی شود
- نگهداری آن دشوار شود
- احتمال خطای برنامه‌نویس افزایش پیدا کند

بنابراین:

Nested Ifرا فقط تا زمانی استفاده کنید که ساختار برنامه را خواناتر می‌کند

۲۴– مثال نامناسب

مثلاً:

```
if age >= 18:
    if is_student:
        if has_card:
            if account_active:
                print("Access granted")
```

این کد از نظر پایتون صحیح است، اما ممکن است بیش از حد تو در تو باشد

گاهی می‌توان آن را به شکل ساده‌تر نوشت:

```
if age >= 18 and is_student and has_card and account_active:
    print("Access granted")
```

اگر همه شرایط اهمیت یکسانی داشته باشند، این روش خواناتر است

۲۵– چه زمانی Nested Ifبهتر است؟

فرض کنید می‌خواهیم پیام‌های متفاوتی برای هر مرحله نمایش دهیم:

```
if username:
    if password:
        if is_active:
```

```
        print("Login successful")
    else:
        print("Account is inactive")
else:
    print("Password is required")
else:
    print("Username is required")
```

اینجا Nested If مناسب است، چون هر مرحله **نتیجه متفاوتی** دارد

۲۶-چه زمانی andبهتر است؟

اگر فقط می‌خواهیم بدانیم آیا تمام شرایط برقرار هستند یا نه، استفاده از andمعمولاً بهتر است:

```
if username and password and is_active:
    print("Login successful")
```

در اینجا نیازی به چند سطح Nested If نداریم

۲۷– یک قانون ساده برای انتخاب

می‌توانیم این قانون را به دانش‌آموزان بدهیم:

اگر می‌گوییم:

همه این شرایط باید برقرار باشند اغلب andمناسب است:

```
if condition1 and condition2:
```

اگر می‌گوییم:

اگر شرط اول برقرار بود، بعد برو شرط دوم را بررسی کن

Nested Ifمی‌تواند مناسب باشد:

```
if condition1:
if condition2:
```

۲۸– یک مثال مقایسه‌ای

روش اول Nested If :

```
age = 22
has_license = True
if age >= 18:
    if has_license:
        print("You can drive")
```

روش دوم and :

```
age = 22
has_license = True
if age >= 18 and has_license:
    print("You can drive")
```

در این مثال هر دو مناسب هستند

اما اگر بخواهیم پیام متفاوتی بدهیم:

```
if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("You are too young")
```

Nested If انتخاب مناسبی است

۲۹– ارتباط Nested If باIndentation

یکی از مهم‌ترین نکات آموزشی این بخش، Indentationاست

مثلاً:

```
if age >= 18:
   if has_license:
      print("You can drive")
```

تعداد فاصله‌ها مشخص می‌کند که کدام دستور متعلق به کدام ifاست

تمرین ۱ رانندگی

برنامه‌ای بنویسید که:

- ابتدا سن را بررسی کند
- اگر سن حداقل ۱۸ بود، بررسی کند آیا گواهینامه دارد
- در نهایت پیام مناسب نمایش دهد

age = 20

has_license = True

خروجی مورد انتظار:

You can drive

تمرین ۲ ورود به سیستم

متغیرهای زیر را در نظر بگیرید:

```
username = "Ali"
password = "1234"
is_active = True
```

با استفاده از Nested If بررسی کنید:

۱. usernameخالی نباشد

۲. passwordخالی نباشد

۳. حساب فعال باشد

تمرین ۳ نمره و حضور

score = 85

attendance = 90

اگر نمره حداقل ۶۰ بود، حضور را بررسی کنید

اگر حضور حداقل ۸۰ بود:

Passed

در غیر این صورت:

Low attendance

و اگر نمره کمتر از ۶۰ بود:

Failed

تمرین ۴ خرید آنلاین

فرض کنید:

balance = 100

price = 80

برنامه‌ای بنویسید که ابتدا بررسی کند آیا موجودی کافی است یا خیر اگر موجودی کافی بود، بررسی کند آیا محصول در انبار موجود است:

in_stock = True

خروجی مناسب نمایش دهید

تمرین ۵ سیستم نمره‌دهی

score = 92

extra_credit = 5

اگر نمره حداقل ۹۰ بود، بررسی کنید امتیاز اضافه وجود دارد یا خیر

نتایج:

A+

A

B

C

را بر اساس شرایط مناسب نمایش دهید

آموزش passدر پایتون

در مباحث قبلی یاد گرفتیم که بعد از if، elif، elseو سایر ساختارهای کنترلی، باید حداقل یک دستور قرار داشته باشد اما گاهی اوقات هنگام طراحی برنامه هنوز نمی‌دانیم قرار است داخل یک بخش چه کدی بنویسیم، یا عمداً می‌خواهیم آن بخش هیچ کاری انجام ندهد

در چنین شرایطی از کلمه کلیدی **pass**استفاده می‌کنیم

۱– pass چیست؟

passیک دستور در پایتون است که هیچ کاری انجام نمی‌دهد یعنی وقتی پایتون به passمی‌رسد، آن را اجرا می‌کند، اما هیچ خروجی یا تغییر خاصی ایجاد نمی‌شود.

مثلاً:

```
a = 33
b = 200
if b > a:
   pass
```

شرط:

200 > 33

درست است، بنابراین پایتون وارد ifمی‌شود و passرا اجرا می‌کند اما passهیچ کاری انجام نمی‌دهد

۲– چرا به passنیاز داریم؟

در پایتون نمی‌توانیم یک بلوک کد را کاملاً خالی بگذاریم مثلاً این کد:

```
if age >= 18:
```

خطای نحوی ایجاد می‌کند، چون پایتون انتظار دارد بعد از :حداقل یک دستور وجود داشته باشد اما اگر هنوز نمی‌خواهیم کاری انجام دهیم، می‌توانیم بنویسیم:

```
if age >= 18:
    pass
```

اکنون کد کاملاً معتبر است

۳– pass به زبان ساده

passیعنی فعلاً هیچ کاری نکن و از این قسمت عبور کن

مثلاً:

```
if age < 18:
    pass
else:
    print("Access granted")
```

اگر سن کمتر از ۱۸ باشد، هیچ کاری انجام نمی‌شود اگر سن ۱۸ یا بیشتر باشد، پیام نمایش داده می‌شود

۴– pass خروجی تولید نمی‌کند

مثلاً:

```
x = 10
if x > 5:
    pass
print("Program finished")
```

شرط درست است، اما passهیچ خروجی‌ای ندارد

تنها خروجی: Program finished

بنابراین:

pass

با:

print()

تفاوت دارد

print()می‌تواند خروجی ایجاد کند، ولی passهیچ کاری انجام نمی‌دهد

۵– pass در مقابل print

مثلاً:

```
if age >= 18:
    pass
```

هیچ چیزی نمایش نمی‌دهد

اما:

```
if age >= 18:
    print("Adult")
```

پیام نمایش داده می‌شود

بنابراین passبرای انجام ندادن هیچ کاری است، نه برای نمایش پیام

۶– مثال ساده

```
age = 20
if age < 18:
    pass
else:
    print("Access granted")
```

چون:

20 < 18 → False

بنابراین elseاجرا می‌شود:

Access granted

۷– pass به عنوانPlaceholder

یکی از مهم‌ترین کاربردهای passاین است که از آن به عنوان جایگاه موقت برای کد آینده استفاده کنیم فرض کنید هنوز منطق مربوط به افراد زیر ۱۸ سال را ننوشته‌ایم:

```
age = 16
if age < 18:
   pass  # TODO: Add underage logic later
else:
   print("Access granted")
```

یعنی:

فعلاً این قسمت خالی است، ولی بعداً قرار است کد مربوط به افراد زیر ۱۸ سال را اضافه کنیم

۹– تفاوت passوComment

این قسمت بسیار مهم است ممکن است بگوئیم اگر نمی‌خواهم داخل ifچیزی بنویسم، چرا کامنت نگذارم؟

مثلاً:

```
if score > 90:
   # Excellent score
```

این کد باعث خطا می‌شود

چرا؟

چون کامنت: # Excellent score برای پایتون دستور محسوب نمی‌شود پایتون همچنان می‌گوید:

داخل این ifهیچ دستوری وجود ندارد

۱۰– passدر if-elif-else

می‌توانیم در هر یک از شاخه‌های if، elifیا elseاز passاستفاده کنیم

مثلاً:

```
value = 50
if value < 0:
   print("Negative value")
elif value == 0:
   pass
else:
   print("Positive value")
```

اگر:

value = 0

باشد، بخش elifاجرا می‌شود و passهیچ کاری انجام نمی‌دهد

۱۳– مثال با چند حالت

فرض کنید برنامه‌ای داریم که وضعیت یک عدد را بررسی می‌کند:

```
number = 0
if number < 0:
   print("Negative")
elif number == 0:
   pass
else:
   print("Positive")
```

برایnumber = 0:

number < 0 → False

number == 0 → True

بنابراین passاجرا می‌شود و هیچ چیزی نمایش داده نمی‌شود

۱۴– آیا passشرط را متوقف می‌کند؟

خیر passفقط **هیچ** کاری انجام نمی‌دهد

مثلاً:

```
x = 10
if x > 5:
   pass
print("Hello")
```

بعد از اجرای pass، برنامه به خط بعدی می‌رود:

Hello

بنابراین passبه معنی برنامه را متوقف کن نیست

۱۵– تفاوت passبا break

این دو را نباید با هم اشتباه گرفت

pass

هیچ کاری انجام نمی‌دهد:

pass

break

از یک حلقه خارج می‌شود:

break

مثلاً:

```
for i in range(5):
   if i == 2:
      pass
   print(i)
```

خروجی:

0

1

2

3

4

اما:

```
for i in range(5):
    if i == 2:
        break
    print(i)
```

خروجی:

0

1

چون break حلقه را متوقف می‌کند

۱۶- تفاوت pass با continue

continue نیز مخصوص حلقه‌هاست و باعث می‌شود اجرای دور فعلی حلقه متوقف شود و حلقه به تکرار بعدی برود

اما:

pass

فقط هیچ کاری انجام نمی‌دهد

مثلاً:

```
for i in range(5):
    if i == 2:
        pass
    print(i)
```

همه اعداد چاپ می‌شوند

۱۷– pass در Function

passفقط برای ifنیست می‌توانیم در تابعی که هنوز پیاده‌سازی نشده است نیز از آن استفاده کنیم.

مثلاً:

```
def calculate_discount(price):
    pass
```

این تابع وجود دارد، اما هنوز کاری انجام نمی‌دهد می‌توانیم بعداً منطق تخفیف را اضافه کنیم

۱۸-مثال Function

```
def login():
    pass
```

در آینده ممکن است بنویسیم:

```
def login():
    username = input("Username: ")
    password = input("Password: ")

    # Login logic
```

بنابراین passدر مراحل اولیه طراحی برنامه بسیار مفید است

۱۹– passدر Class

حتی در تعریف کلاس نیز می‌توانیم از passاستفاده کنیم

مثلاً:

```
class Student:
    pass
```

این کد یک کلاس به نام Studentایجاد می‌کند، اما فعلاً هیچ ویژگی یا متدی داخل آن وجود ندارد

بعداً می‌توانیم آن را کامل کنیم:

```
class Student:
    def __init__(self, name):
        self.name = name
```

۲۰– pass در حلقه‌ها

passمی‌تواند در حلقه forو whileنیز استفاده شود مثلاً:

```
for i in range(5):
    pass
```

حلقه اجرا می‌شود، اما در هر تکرار passهیچ کاری انجام نمی‌دهد

۲۱–یک مثال کاربردی‌تر

فرض کنید می‌خواهیم اعداد ۱ تا ۱۰ را بررسی کنیم، اما فعلاً برای اعداد زوج هیچ کاری انجام ندهیم:

```
for number in range(1, 11):
    if number % 2 == 0:
        pass
    else:
        print(number)
```

در اینجا برای اعداد زوج passاجرا می‌شود

خروجی:

1

3

5

7

9

۲۴–آیا passهمیشه لازم است؟

خیر اگر داخل ifدستور دیگری داشته باشیم، نیازی به passنیست

مثلاً:

```
if age >= 18:
   print("Adult")
```

کاملاً صحیح است

passزمانی کاربرد دارد که واقعاً هیچ دستور دیگری نداریم یا می‌خواهیم یک قسمت را فعلاً خالی نگه داریم

۲۵– مثال نهایی ترکیبی

```
age = 17
if age < 18:
   pass
else:
   print("You can access the system")
print("Program finished")
```

چون:

17 < 18 → True

پس passاجرا می‌شود اما برنامه متوقف نمی‌شود و خط بعدی اجرا می‌شود:

Program finished